

Structures and User-Defined Types

Due at the start of your next lab period

Objectives

Use **struct** to create a data aggregate

Use **const** and **define** statements to improve flexibility and re-usability of functions

Use a **NULL** return value to indicate an error

Requirements

Write a C program that creates Vectors containing numbers and then prints and manipulates those Vectors in various ways.

Your program must:

1) Use **typedef struct** to create a new data type **Vector** to represent a mathematical vector. The data type (WORD) of the numbers contained in the vector must be **#defined** in the header file (along with the print format string **WORD_FORMAT**). The fixed maximum length of the vector must also be **#defined**.

2) The file **vectorMath.h** must contain the following function declarations:

```
/* Perform math on the contents of two vectors and put the result in the
 * third vector.
 * The mathematical operator 'op' must be one of '+', '-', '*', '/'.
 * If the vectors are of incompatible size or if the operator is
 * invalid, the return value is NULL */
```

```
Vector *vectorMath (const Vector v1, const Vector v2,
                    Vector *resultPtr, const char op);
```

```
/* Print the values in a Vector using the print format WORD_FORMAT
 * to the specified stream */
```

```
void printVector (const Vector v, FILE *stream);
```

3) **vectorMath()** must check that the input Vectors have the same size. Return **NULL** if not.

It must also return **NULL** if the operator is invalid. Otherwise, return **result** (which is a pointer that isn't **NULL** because it contains the address of the resulting Vector). Use a **switch** statement for **op**.

4) **printVector()** must use pointer notation with post-increment.

5) After creating the Vectors, the **main()** will then:

a) Print the contents of two input Vectors using **printVector()**

b) Perform the four mathematical operations, printing the result each time, if successful and printing an error message if the operation fails.

c) Demonstrate that the error checking works by trying an operation on two Vectors with incompatible sizes and also by trying an invalid operator.

6) Your project must have two source files and one header file.

7) You must demonstrate that your **#defines** and **consts** work properly by recompiling the program to handle a different data type.

8) Optional: Make **WORD** and **WORD_FORMAT** compiler options. You can make this mandatory or optional (your choice). You will need to use **#ifndef** for this. If it is optional, you will need to specify a default, otherwise you will need to use **#error**.

Note that this is a compile-time option, not a run-time command-line argument.

It does not use **argv** because **WORD** and **WORD_FORMAT** are passed to the compiler. See the HOWTO for this lab on the course lab page.

Evaluation

A working, well-documented program
Optional command-line defines

9/10
1/10

Sample Run

```
dale$ ./vectorMath
Vector #1      1.0000    4.0000   -7.2000   -6.6600    1.2300
Vector #2     -5.2000   11.1000    0.1000    9.9000    7.1230
Sum           -4.2000   15.1000   -7.1000    3.2400    8.3530
Difference      6.2000   -7.1000   -7.3000  -16.5600   -5.8930
Product       -5.2000   44.4000   -0.7200  -65.9340    8.7613
Quotient      -0.1923    0.3604  -72.0000   -0.6727    0.1727
Division failed
Exponentiation failed
```